



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sistema de Monitoreo y Control IoT para Hidroponía

UNIDAD DE APRENDIZAJE

INTERNET DE LAS COSAS|EMBEDDED SYSTEMS

Alumno:

PAREDES MARTINEZ JONATHAN URIEL

Grupo

6CM3

PROFESOR:

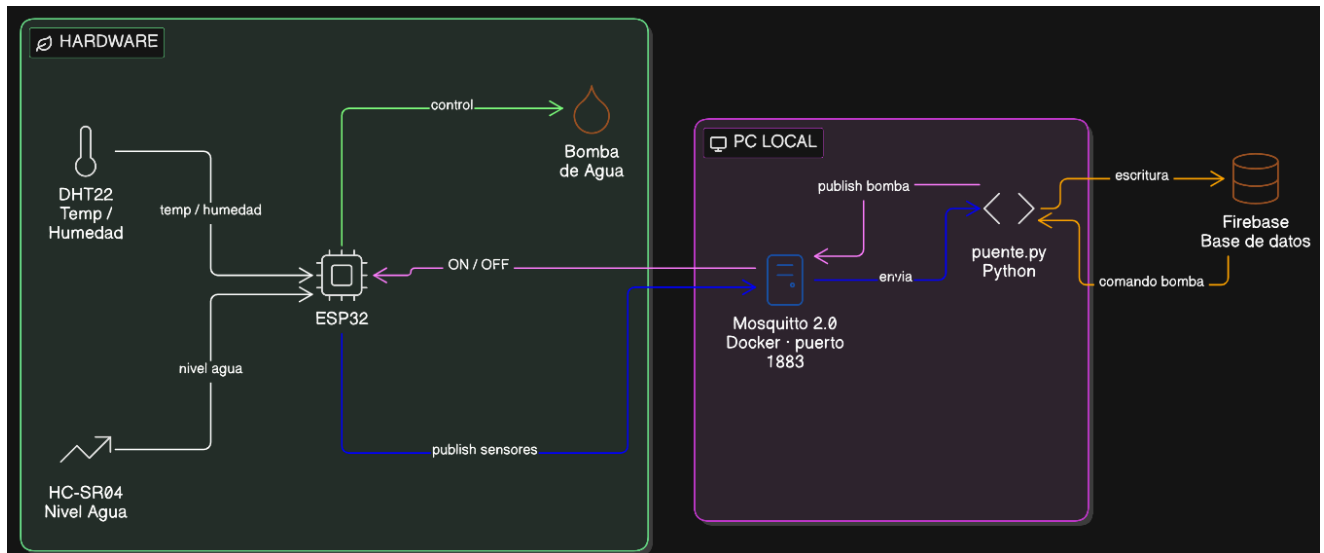
MIGUEL ANGEL ALEMAN ARCE

1. Arquitectura General del Sistema

El sistema implementa una arquitectura IoT híbrida basada en eventos y sincronización en tiempo real. Está diseñado para garantizar una comunicación bidireccional eficiente entre los dispositivos físicos (o simulados) en el borde y las aplicaciones de usuario (web y móvil).

1.1 Flujo de Datos

El intercambio de información se realiza a través de dos flujos principales bien definidos:



1.2 Componentes de la Solución

- **ESP32 (Borde):** Microcontrolador encargado de publicar variables ambientales simuladas de manera realista y de suscribirse al canal de control para operar el estado de la bomba de agua.
- **Mosquitto MQTT Broker:** Intermediario ligero de mensajería (corriendo sobre Docker) que actúa como el punto de entrada de baja latencia para los dispositivos de hardware.
- **Bridge Python (Middleware):** Agente bidireccional encargado de hacer un puente síncrono/asíncrono entre el broker MQTT local y la base de datos en la nube.
- **Firestore Realtime Database:** Base de datos NoSQL en la nube orientada a documentos en tiempo real. Funciona como la "fuente única de verdad" (Single Source of Truth) para todo el estado del sistema.
- **Dashboard React:** Interfaz web responsive que permite visualizar el historial gráfico de las variables y conmutar de forma manual los actuadores.
- **App Android (HidroSync):** Cliente móvil nativo en Kotlin que integra servicios en primer plano para un monitoreo ininterrumpido mediante notificaciones del sistema

1.3 Infraestructura de Red Local

Para las pruebas de integración local y despliegue físico, se utiliza una red de área local inalámbrica con las siguientes credenciales e identidades IP fijas:

- **SSID (Hotspot):** PIXEL
- **Dirección IP del Servidor (PC Broker/Bridge):** 10.150.131.146
- **Dirección IP del ESP32:** 10.150.131.194
- **Puerto por Defecto MQTT:** 1883

2. Broker MQTT — Mosquitto con Docker

El servicio de mensajería utiliza una imagen oficial de **Eclipse Mosquitto** contenerizada para garantizar la portabilidad y aislar los entornos de configuración de red.

2.1 Archivo de Orquestación: docker-compose.yml

services:

mqtt_broker:

image: eclipse-mosquitto:2.0

container_name: mqtt_broker

ports:

- "1883:1883"

- "9001:9001"

volumes:

- ./mosquitto/config:/mosquitto/config

- mosquitto_data:/mosquitto/data

- mosquitto_log:/mosquitto/log

restart: unless-stopped

volumes:

mosquitto_data:

mosquitto_log:

2.2 Archivo de Configuración: mosquitto/config/mosquitto.conf

listener 1883

allow_anonymous true

persistence true

persistence_location /mosquitto/data/

log_dest file /mosquitto/log/mosquitto.log

log_dest stdout

2.3 Comandos de Operación del Contenedor

- **Iniciar servicio (segundo plano):** docker compose up -d
- **Detener servicio:** docker compose down
- **Monitoreo de bitácoras (logs):** docker compose logs -f

2.4 Matriz de Tópicos MQTT

Tópico	QoS	Dirección	Descripción
escom/iot/hidroponia/sensores/temperatura	0	ESP32 → Broker	Publica la temperatura ambiente actual (°C).
escom/iot/hidroponia/sensores/humedad	0	ESP32 → Broker	Publica el porcentaje de humedad relativa (%).
escom/iot/hidroponia/sensores/nivel_agua	1	ESP32 → Broker	Publica el nivel del tanque en centímetros (cm).
escom/iot/hidroponia/actua dores/bomba	2	Broker → ESP32	Recibe comandos de control para encender (ON) o apagar (OFF).

3. Bridge de Integración Python (MQTT → Firebase)

El *Bridge* es un script en Python que actúa como puente de datos. Se encarga de suscribirse a los tópicos de sensores para persistirlos en la nube, y de escuchar cambios en la nube para inyectarlos en el canal MQTT local.

3.1 Estructura del Módulo

- bridge.py — Lógica y bucle principal de ejecución de eventos.
- requirements.txt — Declaración de dependencias del entorno virtual.
- .env — Variables de entorno y configuración de entorno local.
- firebase-credentials.json — Llave privada y credenciales OAuth2 de la cuenta de servicio de Firebase.

3.2 Dependencias (requirements.txt)

```
paho-mqtt==1.6.1
firebase-admin==6.5.0
python-dotenv==1.0.1
```

3.3 Configuración de Entorno (.env)

```
MQTT_BROKER=localhost
MQTT_PORT=1883
MQTT_CLIENT_ID=bridge_hidroponia
FIREBASE_DATABASE_URL=[https://hidroponia-iot-124be-default-rtdb.firebaseio.com](https://hidroponia-iot-124be-default-rtdb.firebaseio.com)
FIREBASE_CREDENTIALS_PATH=./firebase-credentials.json
TOPIC_TEMPERATURA=escom/iot/hidroponia/sensores/temperatura
TOPIC_HUMEDAD=escom/iot/hidroponia/sensores/humedad
TOPIC_NIVEL_AGUA=escom/iot/hidroponia/sensores/nivel_agua
TOPIC_BOMBA=escom/iot/hidroponia/actuadores/bomba
```

3.4 Lógica Interna y Manejo de Versiones

1. **Suscripción MQTT:** Al arrancar, el cliente se conecta al broker local y se suscribe a los tres tópicos de telemetría de sensores (temperatura, humedad, nivel_agua).
2. **Escritura e Historial en Firebase:** Por cada mensaje recibido por MQTT:
 - Actualiza el nodo en tiempo real del sensor: sensores/{sensor} -> { valor, timestamp }
 - Genera un registro histórico incremental usando métodos de inserción recursivos: historial/{sensor}/push({ valor, timestamp })
3. **Listener en Tiempo Real (Firebase a MQTT):** El bridge mantiene una escucha activa (*listener*) en la base de datos sobre el nodo actuadores/bomba. Al modificarse el valor del estado en la nube (ON/OFF), se genera un payload que se publica inmediatamente a través de MQTT al ESP32.
4. **Retrocompatibilidad de Paho MQTT:** El script valida la firma de la librería instalada. Si se detecta una versión superior a la 1.x, adapta la llamada de inicialización del constructor del cliente MQTT dinámicamente, mitigando los problemas frecuentes generados por la API paho-mqtt v2.x.

3.5 Lanzamiento y Diagnóstico

Para ejecutar el servicio manualmente:

```
cd proyecto_iot\bridge
python bridge.py
```

Para verificar de forma aislada la consistencia de los extremos de comunicación (Firebase y Broker), ejecuta:

```
python test_bridge.py
```

4. Base de Datos: Firebase Realtime Database

Toda la persistencia en tiempo real y la sincronización de clientes se gestiona en la nube de Google Firebase mediante una instancia NoSQL reactiva.

- **Identificador de Proyecto:** hidroponia-iot-124be
- **Dirección URL de Instancia:** <https://hidroponia-iot-124be-default-rtdb.firebaseio.com>
- **Consola de Administración:** <https://console.firebase.google.com>

4.1 Estructura del Esquema JSON

```
{
  "sensores": {
    "temperatura": {
      "valor": "23.5",
      "timestamp": "2026-06-20T03:17:19Z"
    },
    "humedad": {
      "valor": "61.4",
      "timestamp": "2026-06-20T03:17:19Z"
    },
    "nivel_agua": {
      "valor": "16.5",
      "timestamp": "2026-06-20T03:17:19Z"
    }
  },
  "actuadores": {
    "bomba": {
      "valor": "ON",
      "timestamp": "2026-06-20T03:16:52Z"
    }
  },
  "historial": {
    "temperatura": {
      "-Nxxx001": { "valor": "23.5", "timestamp": "2026-06-20T03:17:19Z" }
    },
    "humedad": {
      "-Nxxx002": { "valor": "61.4", "timestamp": "2026-06-20T03:17:19Z" }
    },
    "nivel_agua": {
      "-Nxxx003": { "valor": "16.5", "timestamp": "2026-06-20T03:17:19Z" }
    }
  }
}
```

4.2 Reglas de Seguridad (Fase de Desarrollo)

```
{  
  "rules": {  
    ".read": true,  
    ".write": true  
  }  
}
```

4.3 Credenciales del Entorno Web (.env para React/Vite)

Estas variables están expuestas en el cliente para la inicialización segura del SDK de Firebase:

```
VITE_FIREBASE_API_KEY=AlzaSyCpoH0o4PtU9m4kIMrNylmCA8u7ZS3Ow-Q  
VITE_FIREBASE_AUTH_DOMAIN=hidroponia-iot-124be.firebaseio.com  
VITE_FIREBASE_DATABASE_URL=[https://hidroponia-iot-124be-default-rtdb.firebaseio.com](https://h  
idroponia-iot-124be-default-rtdb.firebaseio.com)  
VITE_FIREBASE_PROJECT_ID=hidroponia-iot-124be  
VITE_FIREBASE_STORAGE_BUCKET=hidroponia-iot-124be.firebaseio.com  
VITE_FIREBASE_MESSAGING_SENDER_ID=559868412338  
VITE_FIREBASE_APP_ID=1:559868412338:web:fc8bd368857df42baca4ab
```

5. Firmware ESP32: Simulación y Control

El firmware del microcontrolador está escrito en C++ bajo la API de Arduino. En lugar de usar sensores físicos complejos, simula el comportamiento físico de un sistema hidropónico con transiciones suaves en cada lectura para verificar la viabilidad lógica de la plataforma.

- **Librería Principal Requerida:** PubSubClient (por Nick O'Leary, instalable desde el Gestor de Librerías).
-

5.1 Algoritmo de Simulación del Entorno

Para emular valores reales de manera suave, se utiliza una técnica de generación pseudoaleatoria acotada:

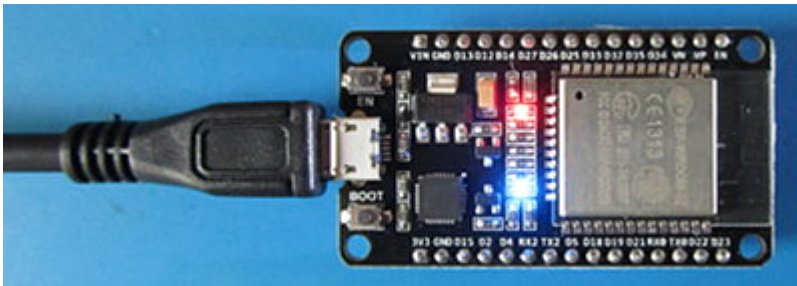
$$\text{Valor_Actual} = \text{Valor_Anterior} + \Delta$$

Donde Δ (Delta) es un diferencial de ruido aleatorio calculado entre un rango específico de tolerancia para simular variaciones físicas reales:

- **Temperatura:** Rango continuo de 16.0 °C a 30.0 °C (Margen normal: 18.0 °C - 26.0 °C).
- **Humedad:** Rango continuo de 45.0% a 75.0% (Margen normal: 50.0% - 70.0%).
- **Nivel de Agua:** Altura medida de 8.0 cm a 32.0 cm (Margen normal: 10.0 cm - 30.0 cm).

5.2 Lógica de Ejecución

1. Verifica conexión WiFi y MQTT (reintento automático).
2. Cada 5 segundos calcula nuevos valores simulados y publica en MQTT.
3. Al recibir comando "ON" en el tópico de la bomba, enciende el LED físico (GPIO 2). Si recibe "OFF", lo apaga.



5.3 Lógica del Ciclo de Ejecución (loop)

1. **Verificación de Enlace:** Evalúa el estado de conexión de la red local WiFi y del broker MQTT. Si la conexión se pierde, inicia un reintento no bloqueante de conexión automática.
2. **Ciclo de Eventos (client.loop()):** Mantiene la escucha de los eventos MQTT entrantes y asocia las llamadas de retorno de comandos (*callbacks*).
3. **Publicación Temporizada (5 segundos):** Mediante variables no bloqueantes basadas en `millis()`, calcula los nuevos estados simulados de los sensores y los emite hacia los tópicos correspondientes con su formato de cadena.
4. **Recepción de Comandos (Control):** Al detectar una carga útil en `escom/iot/hidroponia/actuadores/bomba`:
 - Si es "ON" → `digitalWrite(STATUS_LED, HIGH)` (Enciende el LED físico en placa).
 - Si es "OFF" → `digitalWrite(STATUS_LED, LOW)` (Apaga el LED físico en placa).

5.4 Procedimiento de Carga (Flashing)

1. Conectar la placa de desarrollo ESP32 al PC por medio de un puerto microUSB o USB-C funcional.
2. Abrir el archivo `hidroponia.ino` desde el IDE oficial de Arduino.
3. Configurar la placa destino yendo a **Tools -> Board -> ESP32 Dev Module**.
4. Seleccionar la interfaz física yendo a **Tools -> Port** (Elegir puerto serie correspondiente, por ejemplo, COM3).
5. Subir el programa pulsando `Ctrl+U`.
6. Monitorear el estado físico de la red interna abriendo la herramienta **Serial Monitor** configurando la velocidad de baudios a 115200.

6. Dashboard Web — React + Vite

El Dashboard es una aplicación SPA (Single Page Application) modular diseñada para ejecutarse del lado del cliente, ofreciendo una experiencia responsiva con un tema oscuro moderno.



6.1 Stack Tecnológico

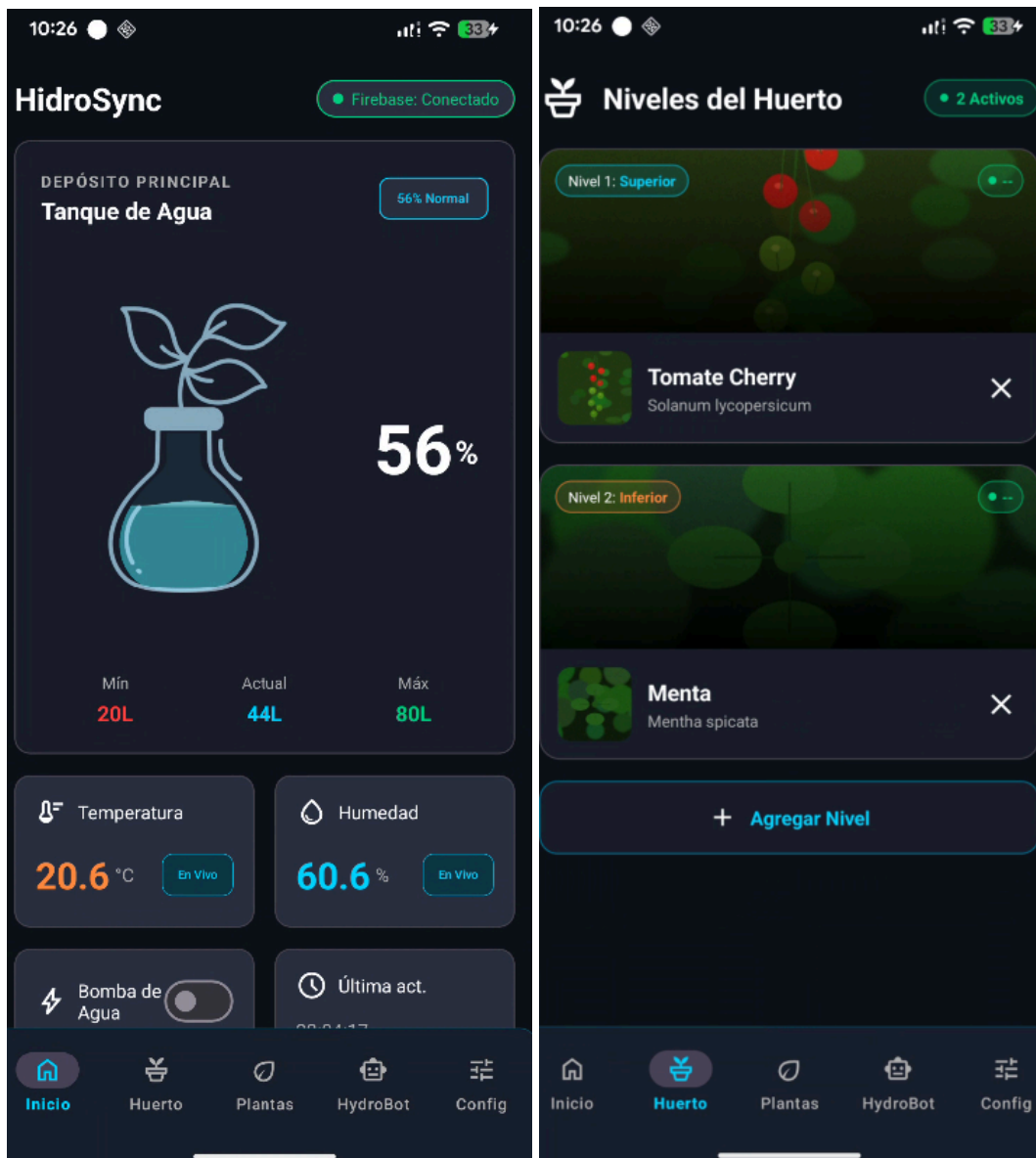
- **Framework de Vista:** React 18 con empaquetador ultrarrápido Vite 5.
- **Conexión en Tiempo Real:** Firebase Client SDK (firebase/database).
- **Visualización de Datos:** Recharts (Gráficas de área vectorizadas).
- **Estilos Gráficos:** Tailwind CSS (Diseñado bajo enfoque móvil con variantes oscuras automáticas).

6.2 Componentes de la Arquitectura del Software

- `src/firebase.js`: Inicializa la app de Firebase utilizando las credenciales seguras del entorno.
- `src/hooks/useSensors.js`: Custom hook que centraliza las conexiones con la API de Firebase. Expone en tiempo real las variables de sensores (`sensores/*`), la lista de historial filtrada para renderizar gráficos (`historial/*`), y provee funciones mutadoras de estado como `setPumpState(estado)` para comandar la bomba.
- `src/components/SensorCard.jsx`: Tarjeta interactiva. Cuenta con umbrales lógicos integrados que cambian dinámicamente el color del borde de la tarjeta según los rangos tolerables del sistema (Verde = Normal, Amarillo = Límite, Rojo = Alarma crítica).
- `src/components/PumpControl.jsx`: Botón interactivo de palanca que sincroniza visualmente el estado físico del LED de la bomba mediante una conmutación inmediata de estado.
- `src/components/SensorChart.jsx`: Gráfica interactiva de alto rendimiento optimizada para mostrar únicamente las últimas 20 lecturas históricas.
- `src/App.jsx`: Contenedor principal de la interfaz de usuario.

7. App Android — HidroSync (Kotlin)

Aplicación móvil nativa estructurada bajo el patrón de arquitectura de diseño de software Android estándar.



7.1 Configuración Base del Proyecto

- **Paquete Base:** com.example.hidroponia
- **SDK Mínimo:** API 34 (Android 14)
- **SDK de Destino:** API 36
- **Maapeo de Dependencias:** Firebase BOM 33.7.0 con la dependencia reactiva integrada firebase-database-ktx.
- **Archivo Obligatorio en Proyecto:** app/google-services.json (Integración a nivel de compilación con los servicios de Google Cloud).

7.2 Conversión de Niveles y Gestión de Unidades

El ESP32 reporta la distancia de nivel de agua en centímetros físicos (cm). La aplicación móvil procesa estos valores en tiempo real convirtiéndolos en métricas legibles para el usuario usando la siguiente relación matemática:

$$\text{Porcentaje de Nivel (\%)} = \left(\frac{\text{Lectura en cm}}{30.0} \right) \times 100$$

Sabiendo que el tanque tiene una capacidad máxima de 80 litros, el volumen se calcula de la siguiente manera:

$$\text{Volumen Disponible (L)} = 80.0 \times \left(\frac{\text{Porcentaje de Nivel}}{100} \right)$$

La interfaz móvil cuenta con un gráfico animado del tipo *matraz o contenedor hidropónico* que oscila visualmente para reflejar la escala del nivel calculado.

7.3 ServicioEstado.kt — Servicio de Primer Plano (Foreground Service)

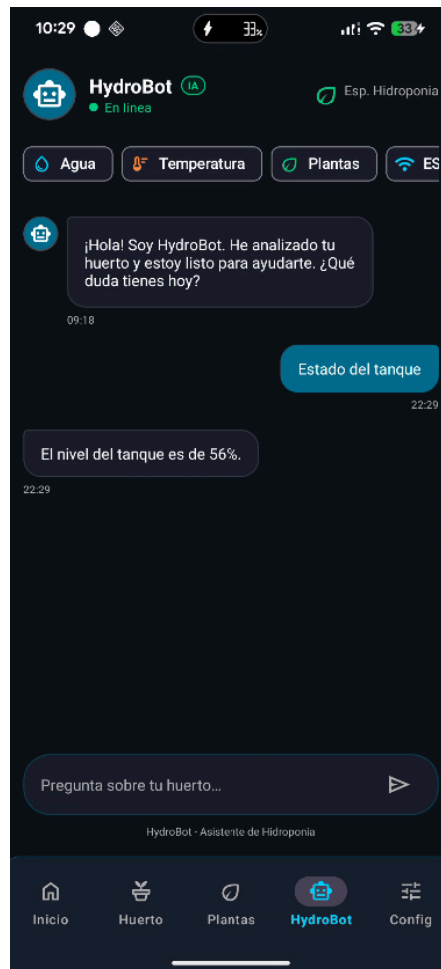
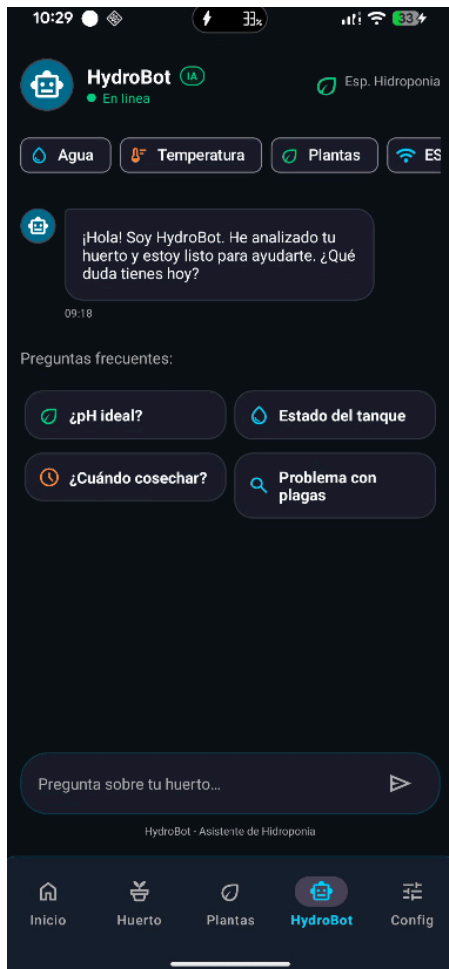
Para mitigar que el sistema operativo suspenda el flujo de datos del huerto cuando el dispositivo móvil esté con la pantalla apagada o en suspensión de recursos, se implementa un **Foreground Service**:

- Mantiene una conexión persistente bidireccional mediante listeners directos con Firebase.
- Muestra un elemento interactivo en la barra de notificaciones del sistema con los valores actualizados al segundo de humedad, temperatura y volumen.
- Dispone de un indicador de conectividad integrado: un distintivo tipo "badge" que cambia entre Verde (Conexión Firebase OK) y Rojo (Pérdida de señal).
- Permite un acceso de encendido/apagado rápido de la bomba mediante acciones directas del sistema de notificaciones sin requerir entrar en la interfaz gráfica de la aplicación móvil.

8. Chat de IA — HydroBot

HydroBot es el asistente virtual experto integrado de forma nativa en los clientes finales para brindar acompañamiento técnico especializado en hidroponía.

- **Módulo Destino:** FragmentoChatbot.kt
- **SDK Utilizado:** Cohere Java SDK v1.10.1



- **Modelo de Lenguaje:** command-r (optimizado para agentes de conversación rápidos con uso de herramientas).

8.1 Inyección de Contexto IoT en Tiempo Real

El bot no responde de forma aislada; es alimentado por un sistema de *RAG (Retrieval-Augmented Generation)* contextual ligero que lee el estado de la base de datos de Firebase por medio del gestor de estados `StateManager.obtenerResumenParaBot()`.

Cuando el usuario inicia una consulta, la aplicación inyecta el siguiente bloque de contexto invisible al prompt original enviado a la API de Cohere:

[CONTEXTO DEL SISTEMA]

El usuario está monitoreando un huerto hidropónico en tiempo real.

Los sensores actuales reportan:

- Temperatura: {temperatura_actual} °C
- Humedad: {humedad_actual} %

- Nivel de Agua en Tanque: {nivel_agua_porcentaje} %
- Estado de Bomba de Agua: {estado_bomba}
- Estado de Enlace: Conectado a Firebase RTDB

Asiste al usuario como "HydroBot". Si detectas valores fuera de los rangos óptimos, sugiérele correcciones específicas de forma amable.

8.2 Preguntas Frecuentes Predefinidas en Interfaz

Para agilizar el flujo de uso móvil, la pantalla del chat incluye chips de interacción rápida para las siguientes consultas:

- *¿Cuál es el rango de pH ideal para mis cultivos?*
- *¿Cómo se encuentra el nivel de agua del tanque actualmente?*
- *¿Cuáles son las plagas más comunes en lechugas hidropónicas y cómo combatirlas?*
- *¿Cada cuánto tiempo debo programar el riego automático de la bomba?*

9. Guía de Arranque y Despliegue del Sistema

Sigue estos pasos en orden secuencial para iniciar el sistema completo de monitoreo y control de hidroponía desde cero.

9.1 Requisitos Previos Obligatorios

- [] Docker Desktop instalado, configurado y activo.
- [] Entorno de ejecución Python 3.11+ configurado en la variable del sistema.
- [] Node.js 18+ instalado.
- [] Archivo firebase-credentials.json posicionado correctamente en proyecto_iot\bridge\.
- [] Placa física de desarrollo ESP32 programada con el firmware hidroponia.ino.

9.2 Paso 1 — Red Local e Identidad de Red

1. Inicializar la red WiFi inalámbrica compartida desde tu smartphone o módem utilizando los valores de SSID: PIXEL y contraseña: 123456789 (valores de ejemplo)
2. Ejecutar el script automatizado para obtener la dirección IP local de tu computador:
C:\get-ip.bat
3. Asegurar que la constante de configuración de red MQTT_BROKER dentro del script hidroponia.ino del ESP32 contenga la dirección IP idéntica a la mostrada en la pantalla del script anterior (por ejemplo: 10.150.131.146).

9.3 Paso 2 — Levantamiento de Servicios del Ecosistema

start-all.bat

El script ejecuta de manera secuencial e independiente las siguientes tareas:

1. **[1/3] Mosquitto Broker:** Inicializa el contenedor Docker de Mosquitto asignando el puerto estándar 1883 para la intercomunicación local de red.
2. **[2/3] Bridge Python:** Lanza una consola que ejecuta el script bridge.py para sincronizar los canales MQTT con la instancia activa de Firebase en tiempo real.
3. **[3/3] Dashboard Web:** Ejecuta en la terminal el servidor local de desarrollo de React (npm run dev) y abre de forma inmediata tu navegador por defecto en la ruta: <http://localhost:5173>.

9.4 Paso 3 — Verificación Visual del Bridge

Observa la ventana de terminal correspondiente al **Bridge MQTT-Firebase**. Deben imprimirse los siguientes registros confirmando la conexión exitosa de los extremos lógicos:

[INFO] Firebase inicializado:

<https://hidroponia-iot-124be-default-rtdb.firebaseio.com>

[INFO] Conectado al broker MQTT localhost:1883

[INFO] Suscrito: escom/iot/hidroponia/sensores/temperatura (QoS 0)

[INFO] Suscrito: escom/iot/hidroponia/sensores/humedad (QoS 0)

[INFO] Suscrito: escom/iot/hidroponia/sensores/nivel_agua (QoS 1)

[INFO] Firebase listener activo: actuadores/bomba

9.5 Paso 4 — Conexión e Inicio del ESP32

1. Conecta el microcontrolador a su fuente de alimentación.
2. Abre la consola de depuración **Serial Monitor** del IDE de Arduino a 115200 baudios.
3. El dispositivo debe inicializar la conexión local arrojando los siguientes eventos informativos:
WiFi OK — IP: 10.150.131.194
Conectando a MQTT... OK
Suscrito: escom/iot/hidroponia/actuadores/bomba
[SIM] Temperatura : 22.3 °C
[SIM] Humedad : 60.5 %
[SIM] Nivel agua : 18.1 cm

Nota: Si se arroja la advertencia de falla recurrente rc=-2, asegúrate de verificar que el broker de Docker de Mosquitto esté corriendo y que la dirección IP configurada en el firmware coincida plenamente.

4.

9.6 Paso 5 — Ejecución de Pruebas Móviles y Cierre del Sistema

1. Ejecuta la aplicación Android HidroSync en tu dispositivo de pruebas o emulador local desde Android Studio.
2. La interfaz móvil se conecta inmediatamente a Firebase utilizando su red celular o conexión a internet de área local, por lo que puede visualizarse el estado del sistema de forma global fuera del perímetro de la red local PIXEL.
3. Para apagar los servicios de forma limpia al terminar las pruebas de depuración, cierra las terminales activas de Python y React, entra en la carpeta raíz del proyecto y ejecuta el siguiente comando terminal:
docker compose down